



ΑΡΙΣΤΟΤΕΛΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΟΝΙΚΗΣ

Πολυτεχνική Σχολή

*Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών
Υπολογιστών*

ΜΙΚΡΟΕΠΕΞΕΡΓΑΣΤΕΣ ΚΑΙ ΠΕΡΙΦΕΡΕΙΑΚΑ

Εργαστήριο 1 - Απρίλιος 2026

ΟΜΑΔΑ 33

-

Λουλουδάκη Αγγελική

AEM : 10976

Γούναρης Χρήστος

AEM : 10638

1 main.c

Η συνάρτηση `main.c` συντονίζει την ροή του προγράμματος. Στην αρχή ορίστηκε το αλφαριθμητικό, εξασφαλίζοντας εύκολη αλλαγή σε περίπτωση `test`, και μετά καλούνται με ορθή σειρά οι συναρτήσεις `assembly`. Αυτές ορίζονται με `extern` έτσι ώστε οι παράμετροι να περάσουν σωστά στους καταχωρητές. Στο τέλος εκτυπώνει με `printf` τα ενδιάμεσα και τελικά αποτελέσματα κάθε συνάρτησης. Η προσθήκη του `while(1)` στο τέλος είναι απαραίτητη για τον μικροελεγκτή.

2 Συνάρτηση `compute_hash.s`

Είναι η πρώτη συνάρτηση και λαμβάνει ως όρισμα το αλφαριθμητικό. Αρχικά, σώζουμε τους καταχωρητές R4-R6 και τον LR (για να ξέρουμε πώς να γυρίσουμε στη C) στη στοίβα. Η διεύθυνση του `pin` αποθηκεύεται στον R0 και εμείς την μεταφέρουμε στο R1, ώστε το τελικό αποτέλεσμα να αποθηκευτεί στο R0. Πρώτα σαρώνουμε το `pin` για να βρούμε τον `null` χαρακτήρα και αποθηκεύουμε το μήκος του στον R2. Έπειτα μεταφέρεται στον R3, ο οποίος αποτελεί την αρχική τιμή του Hash. Μέσα στην `main` λούπα ελέγχεται αν ο κάθε χαρακτήρας είναι κεφαλαίο, πεζό ή ψηφίο και κάνουμε τις αντίστοιχες πράξεις. Κυρίως για τα ψηφία δημιουργήσαμε έναν πίνακα αναφοράς από τον οποίο αντλούνται οι τιμές με χρήση `register offset` ανά 4 bytes (με `LSL #2`). Τέλος αποθηκεύουμε το αποτέλεσμα στο R0 και κάνουμε `pop` ώστε να επαναφέρουμε τις αρχικές τιμές των R4-R6, και να μπει ο παλιός LR μέσα στον PC. Έτσι επιστρέφει αυτόματα ακριβώς εκεί που ήταν στη `main()`.

3 Συνάρτηση `compute_clearance.s`

Η συνάρτηση αυτή λαμβάνει ως όρισμα την τιμή του Hash για να υπολογίσει το επίπεδο πρόσβασης. Πρώτα υπολογίζουμε το `hamming weight` (πόσα 1 υπάρχουν) της τιμής αυτής. Χρησιμοποιούμε το `LSRS` και το `bit` που πάει δεξιά (το λιγότερο σημαντικό `bit`) να αποθηκεύεται στο `Carry Flag`. Προσθέτουμε 1 στον μετρητή R2 μόνο αν το `bit` αυτό ήταν μονάδα χρησιμοποιώντας το `IT CS (If-Then Carry Set)` και την `ADDCS`. Μετά εκτελείται $Amod6 = A - (A/6) * 6$ (όπου A το υπολογισθέν πλήθος άσων) με χρήση του `UDIV`, `MUL` και `SUB` (καθώς δεν υπάρχει έτοιμη εντολή `mod`). Το τελικό αποτέλεσμα -από 0 έως 5- αποθηκεύεται στο R0.

4 Συνάρτηση `lucas_sequence.s`

Η συνάρτηση δέχεται ως όρισμα το αποτέλεσμα του επιπέδου πρόσβασης n . Αρχικά ελέγχουμε αν το n ισούται με μηδέν ή ένα ώστε να το φορτώσουμε κατευθείαν στο R0 και να γλυτώσουμε περιττά `PUSH/POP` (επιστρέφει στην `main` μέσω `BX lr`). Αντίθετα αν $n > 1$, κάνουμε πρώτα `push r4, r5, lr` (είναι σημαντική η αποθήκευση του LR καθώς στην πρώτη αναδρομική κλήση BL θα τον διέγραφε και το πρόγραμμα θα έμπαινε σε ατέρμονο βρόχο. Μετά σώζουμε το τρέχον n στον R4, υπολογίζουμε το $L(n-1)$, αποθηκεύοντας το αποτέλεσμα στον R5, κι καλώ και την $L(n-2)$. Τέλος, αποθηκεύουμε στον R0 το άθροισμα $L(n-1) + L(n-2)$ και το πρόγραμμα επαναφέρει την προηγούμενη κατάσταση των καταχωρητών μέσω `POP`.

5 Συνάρτηση `compute_checksum.s` - BONUS

Η συνάρτηση αυτή σαρώνει το αρχικό αλφαριθμητικό και εφαρμόζει διαδοχική λογική XOR σε δύο στάδια. Πιο συγκεκριμένα φορτώνουμε την τιμή `0xAA` στον R5 έξω από τον βρόχο ώστε να μην επαναλαμβάνεται. Πρώτα κάνουμε XOR τον κάθε χαρακτήρα με το Salt (δεκαεξαδική σταθερά `0xAA`). Το αποτέλεσμα αυτό γίνεται XOR με τον καταχωρητή-”συσσωρευτή” του Checksum που είναι ο R3. Όταν εντοπιστεί ο `null` χαρακτήρας, επιστρέφουμε το αποτέλεσμα στον R0. Όπως και στις προηγούμενες, κάνουμε στο τέλος `POP` για την επαναφορά της κατάστασης των καταχωρητών και την ασφαλή επιστροφή στην `main`.

6 Testing

Για το testing χρησιμοποιήθηκε ο simulator. Πρώτα κάναμε built τους κώδικες και μετά μπαίναμε σε debug mode. Στα διάφορα παράθυρα μπορούσαμε να ελέγχουμε τις τιμές που έπαιρναν οι registers καθώς και οι μεταβλητές μας. Στο debug (printf) viewer μπορούσαμε να δούμε τα εκτυπωμένα αποτελέσματα τις κάθε συνάρτησης. Μάλιστα αλλάξαμε και το αλφαριθμητικό διάφορες φορές ώστε να δούμε κατά πόσο ο κώδικας ανταποκρίνεται ορθά σε κάθε περίπτωση pin.

7 Προβλήματα

Ένα από τα τεχνικά προβλήματα που αντιμετωπίσαμε είναι η χρήση της εντολής πρόσθεσης υπό συνθήκη ADDCS (Add if Carry Set) που χρειαζόμασταν στον υπολογισμό του Hamming Weight. η απευθείας κλήση της εντολής (ADDCS r2, r2, #1) οδηγούσε σε σφάλμα κατά τη μεταγλώττιση. Ψάχνοντας βρήκαμε ότι οι εντολές υπό συνθήκη δεν μπορούν να εκτελεστούν αυτόνομα. Για αυτό χρειάζεται ένα μπλοκ ελέγχου IT (If-Then). Η λύση που βρήκαμε είναι η εξής :

IT CS

ADDCS r2, r2, #1

Μάλιστα προσέξαμε η προηγούμενη εντολή ολίσθησης να περιέχει το 'S' (LSRS), προκειμένου το bit που βγαίνει δεξιά να ενημερώνει ορθά το Carry Flag (C) του συστήματος, και ο μετρητής R2 να αυξάνεται αποκλειστικά όταν εντοπίζεται 1.